

Multi-PDF RAG Application - Architecture & Codebase Explanation

File: app.py - Main Streamlit Application Entry Point

app.py sets up the Streamlit UI, handles file uploads, manages session state, and coordinates the entire RAG pipeline.

- Lines 1-56: Dummy Streamlit fallback for testing without UI.
- Lines 57-83: Imports and Environment Setup (loads API keys, sets GROQ_MODELS).
- Lines 85-190: Streamlit Page Config & Custom CSS styling to provide a dark, modern look.
- Lines 192-200: Session State Initialization (vectorstore, chat_history, etc).
- Lines 202-212: ``get_embeddings()``: Initializes HuggingFace embeddings for dense retrieval.
- Lines 214-223: ``make_splitter()``: Configures RecursiveCharacterTextSplitter for document chunking.
- Lines 225-260: ``process_pdfs()``: Reads uploaded PDFs, redacts PII/secrets via ``pdf_redactor``, splits into chunks, and builds FAISS and BM25 indexes.
- Lines 262-279: ``hybrid_retrieve()``: Implementation of Hybrid Retrieval (FAISS + BM25) combined via Reciprocal Rank Fusion (RRF).
- Lines 281-289: ``build_messages()``: Formats the system prompt, chat history, and context into a conversation array for the LLM.
- Lines 291-374: ``evaluate_response()``: Integrates DeepEval metrics (relevancy, hallucination) using Ollama, with fallback heuristics.
- Lines 376-407: ``get_answer()``: Invokes the Groq LLM across multiple models for redundancy.
- Lines 410-454: Sidebar UI for uploading PDFs and clearing chat history.
- Lines 456-525: Main Chat UI loop. It captures user input, runs ``safe_rag_query`` to fetch the sanitized answer securely, and displays the bot's response along with source citations.

```
from types import SimpleNamespace

try:
    import streamlit as st
except ModuleNotFoundError:
    class SessionState(dict):
        def __getattr__(self, name):
            return self.get(name)
        def __setattr__(self, name, value):
            self[name] = value

    class DummySt:
        session_state = SessionState()
        def set_page_config(self, *args, **kwargs):
            pass
        def markdown(self, *args, **kwargs):
            pass
        def caption(self, *args, **kwargs):
            pass
        def spinner(self, text):
            class _Spinner:
                def __enter__(self):
```

Multi-PDF RAG Application - Architecture & Codebase Explanation

```
        return None
    def __exit__(self, exc_type, exc, tb):
        return False
    return _Spinner()
def cache_resource(self, *args, **kwargs):
    def decorator(func):
        return func
    return decorator
def subheader(self, *args, **kwargs):
    pass
def divider(self, *args, **kwargs):
    pass
def file_uploader(self, *args, **kwargs):
    return []
def slider(self, *args, **kwargs):
    return 8
def button(self, *args, **kwargs):
    return False
def warning(self, *args, **kwargs):
    pass
def info(self, *args, **kwargs):
    pass
def success(self, *args, **kwargs):
    pass
def error(self, *args, **kwargs):
    pass
@property
def sidebar(self):
    return self
def __enter__(self):
    return self
def __exit__(self, exc_type, exc, tb):
    return False
st = DummySt()
import os
import tempfile
import re
from pdf_redactor import redact_text
from dotenv import load_dotenv

try:
    from guard_utils import check_prompt, check_response
except ImportError:
    def check_prompt(prompt: str):
        return True, ""
    def check_response(response: str):
        return True, ""

load_dotenv()
try:
```

Multi-PDF RAG Application - Architecture & Codebase Explanation

```
from deepeval.metrics import AnswerRelevancyMetric, HallucinationMetric, FaithfulnessMetric,
ContextualRecallMetric
except ModuleNotFoundError:
    AnswerRelevancyMetric = HallucinationMetric = FaithfulnessMetric = ContextualRecallMetric = None

GROQ_API_KEY = os.getenv("GROQ_API_KEY", "")

GROQ_MODELS = [
    "llama-3.3-70b-versatile",
    "llama3-70b-8192",
    "mixtral-8b-32768",
]

st.set_page_config(page_title="PDF Chat", page_icon="", layout="wide")

st.markdown("""
<style>
/* Hide Streamlit default header/footer */
#MainMenu, footer, header {visibility: hidden;}

/* Full page dark background */
.stApp { background-color: #212121; }

/* Sidebar */
section[data-testid=\\"stSidebar\\"] {
    background-color: #2f2f2f !important;
}
section[data-testid=\\"stSidebar\\"] * { color: #ececfc !important; }

/* Chat container */
.chat-wrap { max-width: 800px; margin: 0 auto; padding-bottom: 20px; }

/* User bubble */
.user-bubble {
    background: #2f2f2f;
    color: #ececfc;
    border-radius: 18px 18px 4px 18px;
    padding: 12px 18px;
    margin: 8px 0 8px 80px;
    font-size: .95rem;
    line-height: 1.6;
    word-break: break-word;
}

/* Assistant bubble */
.bot-bubble {
    background: #1a1a2e;
    color: #e8e8f0 !important;
    border-left: 3px solid #6c63ff;
    border-radius: 0 18px 18px 18px;

```

Multi-PDF RAG Application - Architecture & Codebase Explanation

```
padding: 14px 18px;
margin: 8px 80px 8px 0;
font-size: .95rem;
line-height: 1.8;
word-break: break-word;
}

/* Source chips */
.src-chip {
  display: inline-block;
  background: #2a2a4a;
  color: #a5b4fc !important;
  border: 1px solid #4f46e5;
  border-radius: 20px;
  padding: 2px 10px;
  font-size: .75rem;
  margin: 3px 3px 0 0;
}

/* Title */
.app-title {
  font-size: 1.6rem;
  font-weight: 800;
  color: #a5b4fc;
  margin-bottom: 4px;
}

.app-sub {
  font-size: .85rem;
  color: #6b7280;
  margin-bottom: 20px;
}

/* Status pill */
.pill-green {
  background:#065f46; color:#6ee7b7 !important;
  border-radius:20px; padding:3px 12px;
  font-size:.78rem; font-weight:700;
}

.pill-gray {
  background:#374151; color:#9ca3af !important;
  border-radius:20px; padding:3px 12px;
  font-size:.78rem; font-weight:700;
}

/* Input box override */
.stChatInput textarea {
  background:#2f2f2f !important;
  color:#ececfc !important;
  border:1px solid #4b5563 !important;
  border-radius:12px !important;
```

Multi-PDF RAG Application - Architecture & Codebase Explanation

```
}

/* Buttons */
.stButton>button {
    border-radius:10px !important;
    font-weight:700 !important;
    background:#4f46e5 !important;
    color:white !important;
    border:none !important;
}
.stButton>button:hover { background:#4338ca !important; }

div[data-testid="stExpander\"] {
    background:#1e1e2e;
    border:1px solid #374151;
    border-radius:8px;
}
</style>
""", unsafe_allow_html=True)

for k, v in [
    ("vectorstore", None),
    ("bm25", None),
    ("all_docs", []),
    ("chat_history", []),
    ("pdf_names", []),
]:
    if k not in st.session_state:
        st.session_state[k] = v

@st.cache_resource(show_spinner="Loading embedding model?")
def get_embeddings():
    try:
        from langchain_huggingface import HuggingFaceEmbeddings
    except ImportError:
        from langchain_community.embeddings import HuggingFaceEmbeddings
    return HuggingFaceEmbeddings(
        model_name="sentence-transformers/all-MiniLM-L6-v2",
        model_kwargs={"device": "cpu"},
        encode_kwargs={"normalize_embeddings": True},
    )

def make_splitter():
    try:
        from langchain_text_splitters import RecursiveCharacterTextSplitter
    except ImportError:
        from langchain.text_splitter import RecursiveCharacterTextSplitter
    return RecursiveCharacterTextSplitter(
        chunk_size=400,
        chunk_overlap=80,
```

Multi-PDF RAG Application - Architecture & Codebase Explanation

```
        separators=["\n\n", "\n", ". ", "? ", "! ", " ", " ", " ", " "],
    )

def process_pdfs(files):
    from langchain_community.document_loaders import PyPDFLoader
    from langchain_community.vectorstores import FAISS
    from rank_bm25 import BM25Okapi

    splitter = make_splitter()
    all_docs, names = [], []

    with tempfile.TemporaryDirectory() as tmp:
        for uf in files:
            path = os.path.join(tmp, uf.name)
            with open(path, "wb") as f:
                f.write(uf.read())
            try:
                pages = PyPDFLoader(path).load()
                if not pages:
                    st.warning(f"No text in `{uf.name}` ? scanned PDF?")
                    continue
                # Redact PII from each page's content before further processing
                for p in pages:
                    p.page_content = redact_text(p.page_content)
                    p.metadata["source_file"] = uf.name
                chunks = splitter.split_documents(pages)
                all_docs.extend(chunks)
                names.append(uf.name)
                st.caption(f"{uf.name} ? {len(chunks)} chunks")
            except Exception as e:
                st.warning(f"Error reading {uf.name}: {e}")

    if not all_docs:
        return None, None, [], []

    vs = FAISS.from_documents(all_docs, get_embeddings())
    tokenized = [doc.page_content.lower().split() for doc in all_docs]
    bm25 = BM25Okapi(tokenized)
    return vs, bm25, all_docs, names

def hybrid_retrieve(question, vectorstore, bm25, all_docs, top_k=8):
    from rank_bm25 import BM25Okapi
    dense_hits = vectorstore.similarity_search(question, k=top_k)
    tokens = question.lower().split()
    bm25_scores = bm25.get_scores(tokens)
    top_bm25_idx = sorted(range(len(bm25_scores)), key=lambda i: bm25_scores[i], reverse=True)[:top_k]
    bm25_hits = [all_docs[i] for i in top_bm25_idx]
    merged = {d.page_content: d for d in dense_hits}
    for d in bm25_hits:
        merged.setdefault(d.page_content, d)
```

Multi-PDF RAG Application - Architecture & Codebase Explanation

```
dense_rank = {d.page_content: i for i, d in enumerate(dense_hits)}
bm25_rank = {all_docs[i].page_content: i for i in top_bm25_idx}
def rrf_score(text):
    dr = dense_rank.get(text, 9999)
    br = bm25_rank.get(text, 9999)
    return 1 / (60 + dr) + 1 / (60 + br)
ranked = sorted(merged.values(), key=lambda d: rrf_score(d.page_content), reverse=True)
return ranked[:top_k]

def build_messages(question, context, chat_history):
    system = """You are a helpful, accurate document assistant.
    Answer based only on the provided document context. Do not hallucinate."""
    messages = [{"role": "system", "content": system}]
    for entry in chat_history[-4:]:
        messages.append({"role": entry["role"], "content": entry["content"]})
    user_msg = f"DOCUMENT CONTEXT:\n{context}\n\nQUESTION: {question}\n\nAnswer based strictly on the context
    above:"
    messages.append({"role": "user", "content": user_msg})
    return messages

def evaluate_response(question: str, answer: str, context: str) -> dict:
    """Run Deepeval metrics on the LLM answer and return a dict of scores."""
    try:
        relevancy = AnswerRelevancyMetric(model="ollama")
    except Exception:
        relevancy = None
    try:
        hallucination = HallucinationMetric(model="ollama")
    except Exception:
        hallucination = None
    try:
        faithfulness = FaithfulnessMetric(model="ollama")
    except Exception:
        faithfulness = None
    try:
        contextual = ContextualRecallMetric(model="ollama")
    except Exception:
        contextual = None

    def _tokens(text: str) -> set:
        return {w.strip('.,!()?[]') for w in text.lower().split() if w}

    if relevancy:
        try:
            relevancy_score = int(relevancy.evaluate(question, answer))
        except Exception:
            relevancy_score = None
    else:
        q_words = _tokens(question)
        a_words = _tokens(answer)
```

Multi-PDF RAG Application - Architecture & Codebase Explanation

```
    relevancy_score = 1 if q_words & a_words else 0

if hallucination:
    try:
        hallucination_score = int(hallucination.evaluate(answer))
    except Exception:
        hallucination_score = None
else:
    if context:
        c_words = _tokens(context)
        overlap = a_words & c_words
        hallucination_score = 1 if len(overlap) / max(len(a_words), 1) > 0.5 else 0
    else:
        hallucination_score = None

if faithfulness:
    try:
        faithfulness_score = int(faithfulness.evaluate(answer, context))
    except Exception:
        faithfulness_score = None
else:
    if context:
        c_words = _tokens(context)
        overlap = a_words & c_words
        faithfulness_score = 1 if len(overlap) / max(len(a_words), 1) > 0.7 else 0
    else:
        faithfulness_score = None

if contextual:
    try:
        contextual_score = int(contextual.evaluate(answer, context))
    except Exception:
        contextual_score = None
else:
    if context:
        c_words = _tokens(context)
        overlap = a_words & c_words
        contextual_score = 1 if len(overlap) / max(len(c_words), 1) > 0.5 else 0
    else:
        contextual_score = None

return {
    "relevancy": relevancy_score,
    "hallucination": hallucination_score,
    "faithfulness": faithfulness_score,
    "contextual_recall": contextual_score,
}

return {
    "relevancy": relevancy_score,
```

Multi-PDF RAG Application - Architecture & Codebase Explanation

```
        "hallucination": hallucination_score,
        "faithfulness": faithfulness_score,
        "contextual_recall": contextual_score,
    }

def get_answer(question, vectorstore, bm25, all_docs, chat_history, top_k=8):
    """Return answer, sources, and the assembled context string used for evaluation."""
    from groq import Groq
    hits = hybrid_retrieve(question, vectorstore, bm25, all_docs, top_k)
    if not hits:
        return "No relevant content found in your PDFs.", []
    sources, ctx_parts = [], []
    for doc in hits:
        src = doc.metadata.get("source_file", "unknown")
        page = doc.metadata.get("page", "?")
        ctx_parts.append(f"[File: {src} | Page: {page}]\n{doc.page_content.strip()}")
        sources.append({"file": src, "page": page})
    context = "\n\n---\n\n".join(ctx_parts)
    messages = build_messages(question, context, chat_history)
    client = Groq(api_key=GROQ_API_KEY)
    last_error = None
    for model in GROQ_MODELS:
        try:
            resp = client.chat.completions.create(
                model=model,
                messages=messages,
                temperature=0,
                max_tokens=100,
            )
            answer_text = resp.choices[0].message.content.strip()
            return answer_text, sources, context
        except Exception as e:
            last_error = str(e)
            if any(x in last_error for x in ["decommissioned", "model_not_found", "404", "not found"]):
                continue
            raise Exception(last_error)
    raise Exception(f"All models failed. Last: {last_error}")

with st.sidebar:
    st.subheader("PDF Chat")
    if st.session_state.pdf_names:
        st.markdown(f'<span class="pill-green">{len(st.session_state.pdf_names)} PDFs ready</span>',
                    unsafe_allow_html=True)
    else:
        st.markdown('<span class="pill-gray">No PDFs loaded</span>', unsafe_allow_html=True)
    st.divider()
    st.subheader("Upload PDFs")
    uploaded = st.file_uploader("Choose PDF files", type=["pdf"], accept_multiple_files=True)
    top_k = st.slider("Retrieval depth", 5, 15, 8, help="Higher = more context per question")
```

Multi-PDF RAG Application - Architecture & Codebase Explanation

```
if st.button("Process PDFs", use_container_width=True):
    if not uploaded:
        st.warning("Upload at least one PDF first.")
    else:
        with st.spinner("Building Hybrid RAG index?"):
            vs, bm25, docs, names = process_pdfs(uploaded)
        if vs:
            st.session_state.vectorstore = vs
            st.session_state.bm25 = bm25
            st.session_state.all_docs = docs
            st.session_state.pdf_names = names
            st.session_state.chat_history = []
            st.success(f"{len(names)} PDFs indexed!")
        else:
            st.error("No text extracted.")
st.divider()
st.subheader("Indexed Files")
if st.session_state.pdf_names:
    for n in st.session_state.pdf_names:
        st.markdown(f"- `{n}`")
    if st.button("Clear Chat & Docs", use_container_width=True):
        for k in ["vectorstore", "bm25", "all_docs", "chat_history", "pdf_names"]:
            st.session_state[k] = None if k in ["vectorstore", "bm25"] else []
        st.rerun()
else:
    st.info("No PDFs yet.")
st.divider()
st.markdown("""
Hybrid RAG:
- BM25 keyword search
- FAISS semantic search
- Reciprocal Rank Fusion
- Conversation memory (4 turns)
Model: llama-3.3-70b
""")

st.markdown('<p class="app-title">PDF Chat</p>', unsafe_allow_html=True)
st.markdown('<p class="app-sub">Ask anything about your uploaded documents</p>', unsafe_allow_html=True)

if st.session_state.vectorstore is None:
    st.markdown("""
<div style="text-align:center; padding:60px 20px; color:#6b7280;">
    <div style="font-size:4rem;">?</div>
    <h3 style="color:#9ca3af;">Upload your PDFs to get started</h3>
    <p>Use the sidebar to upload and process your PDF files</p>
</div>
""", unsafe_allow_html=True)
```

Multi-PDF RAG Application - Architecture & Codebase Explanation

```
else:
    for entry in st.session_state.chat_history:
        if entry["role"] == "user":
            st.markdown(f'<div class="user-bubble">{entry["content"]}</div>', unsafe_allow_html=True)
        else:
            ans = entry["content"]
            ans = re.sub(r'\*\*(.*?)\*\*', r'<b>\1</b>', ans)
            ans = ans.replace("\n", "<br>")
            st.markdown(f'<div class="bot-bubble">{ans}</div>', unsafe_allow_html=True)
            if entry.get("sources"):
                seen = set()
                chips = ""
                for s in entry["sources"]:
                    key = f"{s['file']}|{s['page']}"
                    if key not in seen:
                        seen.add(key)
                        chips += f'<span class="src-chip">{s["file"]} p.{s["page"]}</span>'
                if chips:
                    st.markdown(f'<div style="margin:4px 0 12px 0">{chips}</div>', unsafe_allow_html=True)
question = st.chat_input("Ask a question about your PDFs?")
if question:
    st.markdown(f'<div class="user-bubble">{question}</div>', unsafe_allow_html=True)
    st.session_state.chat_history.append({"role": "user", "content": question, "sources": []})
    prompt_safe, prompt_msg = check_prompt(question)
    if not prompt_safe:
        st.warning(f"Prompt blocked by guard: {prompt_msg}")
    else:
        # Use the secured RAG pipeline with guard layers
        import asyncio
        from secure_rag import safe_rag_query
        result = asyncio.run(safe_rag_query(question))
        if not result.passed:
            st.warning(f"Request blocked: {result.reason}")
            answer = result.sanitized_text or ""
        else:
            answer = result.sanitized_text or ""
        # sources and context are managed inside the guard pipeline
        sources = []
        context = ""

        response_safe, response_msg = check_response(answer)
        if not response_safe:
            st.warning(f"Response blocked by guard: {response_msg}")
            answer = response_msg
        ans_html = re.sub(r'\*\*(.*?)\*\*', r'<b>\1</b>', answer)
        ans_html = ans_html.replace("\n", "<br>")
        st.markdown(f'<div class="bot-bubble">{ans_html}</div>', unsafe_allow_html=True)
    if sources:
        seen, chips = set(), ""
        for s in sources:
```

Multi-PDF RAG Application - Architecture & Codebase Explanation

```
key = f"{s['file']}|{s['page']}"
if key not in seen:
    seen.add(key)
    chips += f'<span class="src-chip">{s["file"]} p.{s["page"]}</span>'
st.markdown(f'<div style="margin:4px 0 12px 0">{chips}</div>', unsafe_allow_html=True)
```

File: secure_rag.py - Secure RAG Pipeline Coordinator

This file orchestrates the security guardrails wrapping the LLM call.

- Lines 1-15: Imports guard layers (LLMGuardLayer, OllamaGuardLayer, GuardrailsLayer, TruffleHogLayer).
- Lines 17-21: ``run_sync()``: A helper to run synchronous functions inside the asyncio thread pool.
- Lines 23-32: ``_retrieve_chunks()``: Wraps the hybrid retrieval function.
- Lines 34-44: ``_scan_chunks()``: Scans all retrieved document chunks for prompt injections to prevent indirect attacks.
- Lines 46-60: ``_call_llm()``: Wrapper for the actual LLM completion.
- Lines 62-115: ``safe_rag_query()``: The core secured flow. It scans input using TruffleHog & LLM-Guard, retrieves chunks, scans chunks for injection, formats the prompt, fetches the answer, and finally scans the output using LLM-Guard, TruffleHog, and Guardrails.ai before returning the safe string.

```
import asyncio
import os
from dotenv import load_dotenv
from typing import List, Tuple
from guards.base import GuardResult, log_violation
from guards.llm_guard_layer import LLMGuardLayer
from guards.ollama_guard_layer import OllamaGuardLayer
from guards.guardrails_layer import GuardrailsLayer
from guards.trufflehog_layer import TruffleHogLayer

load_dotenv()

_llm_guard = LLMGuardLayer()
_ollama_guard = OllamaGuardLayer()
_trufflehog = TruffleHogLayer()
_guardrails = GuardrailsLayer(
    fallback_msg=os.getenv("GUARD_FALLBACK_MSG", "Sorry, I can't answer that.")
)

async def run_sync(fn, *args, **kwargs):
    loop = asyncio.get_event_loop()
    return await loop.run_in_executor(None, lambda: fn(*args, **kwargs))

async def _retrieve_chunks(
    question: str, vectorstore, bm25, all_docs, top_k: int = 8
```

Multi-PDF RAG Application - Architecture & Codebase Explanation

```
) -> List[Tuple[str, dict]]:
    from rag_utils import hybrid_retrieve

    hits = await run_sync(hybrid_retrieve, question, vectorstore, bm25, all_docs, top_k)
    return [(hit.page_content, hit.metadata) for hit in hits]
```

```
async def _scan_chunks(chunks: List[Tuple[str, dict]]) -> GuardResult:

    for text, _ in chunks:
        result = await _llm_guard.scan_input(text)
        if not result.passed:
            log_violation(
                category="ChunkInjection", scanner_name="LLMGuardChunk", input_text=text
            )
            return GuardResult(
                passed=False, blocked_by="LLMGuardChunk", reason=result.reason
            )
    return GuardResult(passed=True)
```

```
async def _call_llm(question: str, context: str, chat_history: List[dict]):

    from rag_utils import get_answer

    answer, sources, used_context = await run_sync(
        get_answer,
        question,
        None,
        None,
        None,
        [],
        top_k=8,
    )
    return answer, sources, used_context
```

```
async def safe_rag_query(user_input: str) -> GuardResult:

    # 1. TruffleHog input scanning
    result = await _trufflehog.scan_input(user_input)
    if not result.passed:
        return result

    # 2. LLM?guard input scanning
    result = await _llm_guard.scan_input(user_input)
    if not result.passed:
        return result

    result = await _ollama_guard.scan_input(user_input)
```

Multi-PDF RAG Application - Architecture & Codebase Explanation

```
if not result.passed:
    return result

import streamlit as st

vectorstore = st.session_state.get("vectorstore")
bm25 = st.session_state.get("bm25")
all_docs = st.session_state.get("all_docs")
if not vectorstore or not bm25 or not all_docs:
    return GuardResult(passed=False, blocked_by="RAG", reason="Index not built")

chunks = await _retrieve_chunks(user_input, vectorstore, bm25, all_docs)

chunk_scan = await _scan_chunks(chunks)
if not chunk_scan.passed:
    return chunk_scan

context_parts = []
for text, meta in chunks:
    src = meta.get("source_file", "unknown")
    page = meta.get("page", "?")
    context_parts.append(f"[File: {src} | Page: {page}]\n{text.strip()}")
context = "\n\n---\n\n".join(context_parts)

from rag_utils import get_answer

answer, sources, _ = await run_sync(
    get_answer,
    user_input,
    vectorstore,
    bm25,
    all_docs,
    st.session_state.get("chat_history", []),
    top_k=8,
)

result = await _llm_guard.scan_output(answer, prompt=user_input)
if not result.passed:
    return result
safe_answer = result.sanitized_text

# 6.5 TruffleHog output scanning
result = await _trufflehog.scan_output(safe_answer, prompt=user_input)
if not result.passed:
    return result

result = await _guardrails.validate(safe_answer)
return result
```


Multi-PDF RAG Application - Architecture & Codebase Explanation

```

        secrets_found = [item.get("rule", {}).get("message", "Unknown Secret") for item
in output]

        return False, f"Secrets found: {' ', ' '.join(secrets_found)}"
    except Exception as e:
        pass
    except Exception as e:
        import logging
        logging.error(f"TruffleHog error: {e}")

    return True, ""

is_valid, reason = await loop.run_in_executor(None, _call)

if not is_valid:
    log_violation(category=reason, scanner_name=f"TruffleHog({context})", input_text=text)
    return GuardResult(passed=False, blocked_by="TruffleHog", reason=reason, sanitized_text=None)

return GuardResult(passed=True, sanitized_text=text)

async def scan_input(self, prompt: str) -> GuardResult:
    return await self.scan_text(prompt, context="Input")

async def scan_output(self, response: str, prompt: str = "") -> GuardResult:
    return await self.scan_text(response, context="Output")
```

File: guards/llm_guard_layer.py - LLM-Guard Security Layer

Configures the local LLM-Guard scanners for PII, Toxicity, and Prompt Injection.

- Lines 1-14: Imports necessary scanners from the `llm\_guard` package (fallback to None if unavailable).
- Lines 16-48: `\_\_init\_\_():` Sets up `input\_scanners` (PromptInjection, BanTopics, Toxicity, Anonymize) and `output\_scanners` (OutputToxicity, Sensitive, OutputBanTopics). The Sensitive scanner is explicitly configured with 'DATE_TIME' to catch DOBs.
- Lines 50-71: `\_run\_scanner():` Executes the scanner synchronously in a thread pool. It handles signature checking to pass the 'prompt' correctly to output scanners.
- Lines 73-94: `scan\_input()` / `scan\_output():` Iterates through all registered scanners. If any scanner fails, it aborts the process and logs the violation.

```
import asyncio
from typing import Tuple

# Attempt to import llm_guard scanners; fallback to None
try:
    from llm_guard.vault import Vault
    from llm_guard.input_scanners import PromptInjection, BanTopics, Toxicity, Secrets, Anonymize,
BanSubstrings
    from llm_guard.input_scanners.anonymize import DEFAULT_ENTITY_TYPES
    from llm_guard.output_scanners import Toxicity as OutputToxicity, Sensitive, BanTopics as OutputBanTopics
```

Multi-PDF RAG Application - Architecture & Codebase Explanation

```
except Exception as e:
    import logging
    logging.warning(f"Failed to import llm_guard scanners: {e}")
    Vault = PromptInjection = BanTopics = Toxicity = Secrets = Anonymize = BanSubstrings = None
    DEFAULT_ENTITY_TYPES = []
    OutputToxicity = Sensitive = OutputBanTopics = None

from .base import GuardResult, log_violation

class LLMGuardLayer:
    """Wrapper for llm_guard input and output scanners.

    All methods are async to fit the overall pipeline but the underlying
    scanners are synchronous, so they are executed in a thread pool.
    """

    def __init__(self):
        # Initialize scanners if available
        self.input_scanners = []
        if PromptInjection:
            self.input_scanners.append(('PromptInjection', PromptInjection(threshold=0.5)))
        if BanTopics:
            self.input_scanners.append(('BanTopics', BanTopics(topics=["violence", "weapons"])))
        if Toxicity:
            self.input_scanners.append(('Toxicity', Toxicity(threshold=0.5)))
        if Anonymize and Vault:
            vault = Vault()
            self.input_scanners.append(('Anonymize', Anonymize(vault)))
        if BanSubstrings:
            self.input_scanners.append(('BanSubstrings', BanSubstrings(substrings=["cookware", "weapon",
"harms", "attack"])))

        self.output_scanners = []
        if OutputToxicity:
            self.output_scanners.append(('Toxicity', OutputToxicity(threshold=0.5)))
        if Sensitive:
            self.output_scanners.append(('Sensitive', Sensitive(entity_types=DEFAULT_ENTITY_TYPES +
["DATE_TIME"])))
        if OutputBanTopics:
            self.output_scanners.append(('BanTopics', OutputBanTopics(topics=["violence", "weapons"])))

    async def _run_scanner(self, scanner_callable, text: str, prompt: str = "") -> Tuple[bool, str, str]:
        """Execute a scanner in a thread pool.
        Returns (is_valid, reason, sanitized_text).
        """
        loop = asyncio.get_event_loop()
        # The scanner's .scan() returns (sanitized, is_valid, risk_score) for most
        def _call():
            try:
                import inspect
```

Multi-PDF RAG Application - Architecture & Codebase Explanation

```
sig = inspect.signature(scanner_callable.scan)
if 'output' in sig.parameters:
    result = scanner_callable.scan(prompt, text)
else:
    result = scanner_callable.scan(text)
if isinstance(result, tuple) and len(result) == 3:
    sanitized, is_valid, risk = result
    reason = f"risk {risk:.2f}" if not is_valid else ""
else:
    # Fallback for scanners that just return bool
    is_valid = bool(result)
    sanitized = text
    reason = ""
return is_valid, reason, sanitized
except Exception as e:
    # On unexpected error treat as safe but log
    return True, f"error: {e}", text
return await loop.run_in_executor(None, _call)

async def scan_input(self, prompt: str) -> GuardResult:
    if not self.input_scanners:
        return GuardResult(passed=True, sanitized_text=prompt)
    for name, scanner in self.input_scanners:
        is_valid, reason, sanitized = await self._run_scanner(scanner, prompt)
        if not is_valid:
            log_violation(category=reason or name, scanner_name=name, input_text=prompt)
            return GuardResult(passed=False, blocked_by=name, reason=reason, sanitized_text=None)
        # Update prompt for next scanner
        prompt = sanitized
    return GuardResult(passed=True, sanitized_text=prompt)

async def scan_output(self, response: str, prompt: str = "") -> GuardResult:
    if not self.output_scanners:
        return GuardResult(passed=True, sanitized_text=response)
    for name, scanner in self.output_scanners:
        is_valid, reason, sanitized = await self._run_scanner(scanner, response, prompt=prompt)
        if not is_valid:
            log_violation(category=reason or name, scanner_name=name, input_text=response)
            return GuardResult(passed=False, blocked_by=name, reason=reason, sanitized_text=None)
        response = sanitized
    return GuardResult(passed=True, sanitized_text=response)
```

File: guards/ollama_guard_layer.py - Ollama Llama-Guard-3 Layer

Provides secondary safety checks using an Ollama-hosted model.

- Lines 1-21: Attempts to import `ollama\_guard`.
- Lines 23-39: `\_run\_guard()`: Synchronous thread-pool execution of `self.guard.check(text)`. Returns

Multi-PDF RAG Application - Architecture & Codebase Explanation

whether the text is unsafe.

- Lines 41-47: ``scan_input()``: Uses the guard to block inappropriate questions or prompt injections before they reach the main LLM.

```
import asyncio
import json
from typing import Tuple

# Attempt to import ollama_guard client; fallback to None
try:
    from ollama_guard import Guard as OllamaGuard
except Exception:
    OllamaGuard = None

from .base import GuardResult, log_violation

class OllamaGuardLayer:
    """Wrapper for Ollama Guard (Llama?Guard?3) scanning.

    The guard is invoked after llm?guard input scanning and before the LLM call.
    It returns a GuardResult indicating pass/fail.
    """

    def __init__(self):
        if OllamaGuard:
            try:
                self.guard = OllamaGuard()
            except Exception:
                self.guard = None
        else:
            self.guard = None

    async def _run_guard(self, text: str) -> Tuple[bool, str]:
        """Run the Ollama Guard model in a thread pool.
        Returns (is_safe, category). If guard unavailable, returns (True, "").
        """
        loop = asyncio.get_event_loop()
        def _call():
            if not self.guard:
                return True, ""
            try:
                # The guard's .check() method is hypothetical; adjust if actual API differs
                result = self.guard.check(text)
                # Expected result format: {"unsafe": bool, "category": "S1"}
                unsafe = result.get('unsafe', False)
                category = result.get('category', '')
                return not unsafe, category
            except Exception as e:
                # On error treat as safe but log warning
                return True, ""
        return loop.run_in_executor(None, _call)
```

Multi-PDF RAG Application - Architecture & Codebase Explanation

```
        return await loop.run_in_executor(None, _call)

    async def scan_input(self, prompt: str) -> GuardResult:
        is_safe, category = await self._run_guard(prompt)
        if not is_safe:
            log_violation(category=category or "OllamaGuard", scanner_name="OllamaGuard", input_text=prompt)
            return GuardResult(passed=False, blocked_by="OllamaGuard", reason=category, sanitized_text=None)
        return GuardResult(passed=True, sanitized_text=prompt)
```

File: guards/guardrails_layer.py - Guardrails AI Layer

Validates the LLM output using specific structured expectations.

- Lines 1-17: Imports Guardrails and specific validators (CompetitorCheck, ToxicLanguage).
- Lines 19-27: Sets up the Guard object with the rules.
- Lines 29-41: `\_run\_guard()`: Parses the LLM output against the guardrails rules.
- Lines 43-49: `validate()`: Async wrapper. Returns the fallback message if validation fails.

```
import asyncio
from typing import Tuple

# Attempt to import guardrails; fallback to None
try:
    from guardrails import Guard
    from guardrails.validators import ToxicLanguage, DetectPII, RestrictToTopic
except Exception:
    Guard = None
    ToxicLanguage = DetectPII = RestrictToTopic = None

from .base import GuardResult, log_violation

class GuardrailsLayer:
    """Wrapper for Guardrails AI validators.

    The validators are applied to the final LLM response. On failure a safe
    fallback message is returned.
    """

    def __init__(self, fallback_msg: str = "Sorry, I can't answer that."):
        self.fallback_msg = fallback_msg
        if Guard and ToxicLanguage and DetectPII and RestrictToTopic:
            self.guard = Guard(
                validators=[
                    ToxicLanguage(threshold=0.5, validation_method="sentence"),
                    DetectPII(pii_entities=["EMAIL_ADDRESS", "PHONE_NUMBER", "CREDIT_CARD", "US_SSN"]),
                    RestrictToTopic(
                        valid_topics=["AI", "science", "history", "general knowledge"],
                        disable_classifier=False,
```

Multi-PDF RAG Application - Architecture & Codebase Explanation

```
        disable_llm=True,
    ),
]
)
else:
    self.guard = None

def _run_guard(self, text: str) -> Tuple[bool, str]:
    """Run Guardrails validation synchronously.
    Returns (is_valid, reason). If guard unavailable, returns (True, "")."""
    if not self.guard:
        return True, ""
    try:
        result = self.guard.validate(text)
        passed = result.get('passed', True)
        reason = result.get('reason', '')
        return passed, reason
    except Exception:
        return True, ""

async def validate(self, response: str) -> GuardResult:
    is_valid, reason = self._run_guard(response)
    if not is_valid:
        log_violation(category=reason or "Guardrails", scanner_name="Guardrails", input_text=response)
        return GuardResult(passed=False, blocked_by="Guardrails", reason=reason,
sanitized_text=self.fallback_msg)
    return GuardResult(passed=True, sanitized_text=response)
```

File: rag_utils.py - Core RAG Algorithms

Contains the isolated logic for Hybrid Retrieval and LLM interaction, decoupled from Streamlit.

- Lines 1-26: `hybrid_retrieve()`: Executes FAISS semantic search and BM25 sparse keyword search, computing scores using Reciprocal Rank Fusion ($RRF = 1/(60+rank)$).
- Lines 28-76: `get_answer()`: Uses `hybrid_retrieve` to get chunks, formats a context string with source files/pages, builds the OpenAI-compatible message array with conversation history, and invokes the Groq client with a fallback model chain (llama3-70b-8192 -> mixtral-8b -> etc).

```
import re
import os
from typing import List, Tuple

def hybrid_retrieve(question: str, vectorstore, bm25, all_docs, top_k: int = 8) -> List:

    from rank_bm25 import BM25Okapi
```

Multi-PDF RAG Application - Architecture & Codebase Explanation

```
dense_hits = vectorstore.similarity_search(question, k=top_k)
tokens = question.lower().split()
bm25_scores = bm25.get_scores(tokens)
top_bm25_idx = sorted(
    range(len(bm25_scores)), key=lambda i: bm25_scores[i], reverse=True
)[:top_k]
bm25_hits = [all_docs[i] for i in top_bm25_idx]
merged = {d.page_content: d for d in dense_hits}
for d in bm25_hits:
    merged.setdefault(d.page_content, d)
dense_rank = {d.page_content: i for i, d in enumerate(dense_hits)}
bm25_rank = {all_docs[i].page_content: i for i in top_bm25_idx}

def rrf_score(text):
    dr = dense_rank.get(text, 9999)
    br = bm25_rank.get(text, 9999)
    return 1 / (60 + dr) + 1 / (60 + br)

ranked = sorted(
    merged.values(), key=lambda d: rrf_score(d.page_content), reverse=True
)
return ranked[:top_k]

def get_answer(
    question: str, vectorstore, bm25, all_docs, chat_history: List[dict], top_k: int = 8
):

    from groq import Groq

    hits = hybrid_retrieve(question, vectorstore, bm25, all_docs, top_k)
    if not hits:
        return "No relevant content found in your PDFs.", [], ""
    sources, ctx_parts = [], []
    for doc in hits:
        src = doc.metadata.get("source_file", "unknown")
        page = doc.metadata.get("page", "?")
        ctx_parts.append(f"[File: {src} | Page: {page}]\n{doc.page_content.strip()}")
        sources.append({"file": src, "page": page})
    context = "\n\n---\n\n".join(ctx_parts)
    system = """"You are a helpful, accurate document assistant. \nAnswer based only on the provided document
context. Do not hallucinate."""
    messages = [{"role": "system", "content": system}]
    for entry in chat_history[-4:]:
        messages.append({"role": entry["role"], "content": entry["content"]})
    messages.append(
        {
            "role": "user",
            "content": f"DOCUMENT CONTEXT:\n{context}\n\nQUESTION: {question}\n\nAnswer based strictly on the
```

Multi-PDF RAG Application - Architecture & Codebase Explanation

```
context above:",
    }
)
client = Groq(api_key=os.getenv("GROQ_API_KEY"))
for model in ["llama-3.3-70b-versatile", "llama3-70b-8192", "mixtral-8b-32768"]:
    try:
        resp = client.chat.completions.create(
            model=model, messages=messages, temperature=0, max_tokens=100
        )
        answer = resp.choices[0].message.content.strip()
        return answer, sources, context
    except Exception as e:
        if any(
            x in str(e)
            for x in ["decommissioned", "model_not_found", "404", "not found"]
        ):
            continue
        raise
raise Exception("All models failed.")
```

File: guard_utils.py - Legacy Security Utils

Contains the old heuristic security methods (fallback if advanced layers fail).

- Lines 1-21: Configures legacy static lists and basic LLM-Guard checks (now superseded by the OOP `guards/` module).
- Lines 23-28: `\_BLACKLIST\_PATTERN`: A regex blocking specific violent terms (weapons, knives, etc).
- Lines 30-68: `check\_prompt()` and `check\_response()` functions using the old implementation.

```
import re
import logging

try:
    from llm_guard import scan_prompt, scan_output
    from llm_guard.input_scanners import PromptInjection, Toxicity, Secrets, Anonymize
    from llm_guard.input_scanners.prompt_injection import MatchType
    from llm_guard.output_scanners import Toxicity as OutputToxicity, Sensitive

    _input_scanners = [
        PromptInjection(threshold=0.5, match_type=MatchType.FULL),
        Toxicity(threshold=0.5),
        Secrets(),
        Anonymize(),
    ]

    _output_scanners = [
        OutputToxicity(threshold=0.5),
        Sensitive(),
    ]
```

Multi-PDF RAG Application - Architecture & Codebase Explanation

```
]
_LLM_GUARD_AVAILABLE = True
except Exception as e:
    logging.warning(f"Failed to load LLM Guard scanners: {e}")
    _LLM_GUARD_AVAILABLE = False

_BLACKLIST_PATTERN = re.compile(
    r"\b(?:cookware|glass\s*bottles?|ceramic|knife|blade|weapon|gun|firearm|explosive|molotov|"
    r"improvised\s+weapon|attack|injure|harm)\b",
    re.IGNORECASE,
)

def check_prompt(prompt: str):

    if not _LLM_GUARD_AVAILABLE:
        return True, prompt

    try:
        sanitized_prompt, results_valid, results_score = scan_prompt(
            _input_scanners, prompt
        )
        if all(results_valid.values()):
            return True, sanitized_prompt
        else:
            blocked_by = [k for k, v in results_valid.items() if not v]
            return (
                False,
                f"Prompt blocked by LLM Guard scanners: {' '.join(blocked_by)}",
            )
    except Exception as e:
        logging.error(f"Error in prompt scan: {e}")
        return True, prompt

def check_response(response: str, prompt: str = ""):

    if _BLACKLIST_PATTERN.search(response):
        return (
            False,
            "Response blocked: content violates policy (disallowed weapon?related terms).",
        )

    if not _LLM_GUARD_AVAILABLE:
        return True, response

    try:
        sanitized_response, results_valid, results_score = scan_output(
            _output_scanners, prompt, response
        )
```

Multi-PDF RAG Application - Architecture & Codebase Explanation

```
if all(results_valid.values()):
    return True, sanitized_response
else:
    blocked_by = [k for k, v in results_valid.items() if not v]
    return (
        False,
        f"Response blocked by LLM Guard scanners: {' ', ' '.join(blocked_by)}",
    )
except Exception as e:
    logging.error(f"Error in response scan: {e}")
    return True, response
```

File: pdf_redactor.py - PDF PII Redaction

Sanitizes PDF text immediately after loading to prevent PII ingestion.

- Lines 1-13: Imports PyMuPDF (`fitz`) and defines static regexes for Email, Phone, Credit Cards, and Addresses.
- Lines 15-28: `redact\_text()`: Uses `re.sub` to replace matched PII patterns with `[REDACTED\_EMAIL]`, `[REDACTED\_PHONE]`, etc.
- Lines 30-41: `extract\_and\_redact()`: Helper that extracts raw text from PDF pages and runs `redact\_text` over each chunk.

```
'''pdf_redactor.py
Extracts text from PDF pages and runs the centralised redaction engine
so that PII/secrets are stripped before being indexed into the vector store.
'''

import re
from typing import List

try:
    import fitz # PyMuPDF
except Exception:
    fitz = None

# Import the centralised redaction function so we have one source of truth
from security import redact_text

def extract_text_from_pdf(pdf_path: str) -> List[str]:
    """Extract raw text from each page of a PDF file."""
    if fitz is None:
        raise ImportError('PyMuPDF (fitz) is required for PDF extraction')
    doc = fitz.open(pdf_path)
    pages_text = []
    for page_num in range(doc.page_count):
        page = doc.load_page(page_num)
        pages_text.append(page.get_text())
```

Multi-PDF RAG Application - Architecture & Codebase Explanation

```
return pages_text
```

```
def extract_and_redact(pdf_path: str) -> List[str]:
    """Extract text from a PDF and redact all PII/secrets on each page.
    Returns a list of sanitised text chunks (one per page).
    """
    raw_chunks = extract_text_from_pdf(pdf_path)
    return [redact_text(chunk) for chunk in raw_chunks]
```

File: security.py - General Sanitization Utils

Legacy module for basic string sanitization and masking.

- Lines 1-7: Regex patterns for API keys and passwords.
- Lines 9-35: `sanitize_text()`: Obfuscates passwords, emails (showing only first character), and phone numbers (masking all but last 4 digits).

```
'''security.py
Centralised redaction engine. Catches PII, secrets, and sensitive identifiers
before they are ever shown to the user.
'''
import re

# ??? Email ?????????????????????????????????????????????????????????????
_EMAIL = re.compile(r'[a-zA-Z0-9_+-.]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-]+\.')

# ??? Phone numbers (international + US formats) ?????????????????????????
_PHONE = re.compile(
    r'(\+?\d{1,3}[\s\-])?'
    r'(\(?\d{3}\)?[\s\-]?\d{3}[\s\-]?\d{4})'
)

# ??? Credit / Debit card numbers ?????????????????????????????????????
_CARD = re.compile(r'\b(?:\d[ -]*){13,19}\b')
_CVV = re.compile(r'(?i)(cvv|cvc|security\s*code)\s*[:=\s]\s*\d{3,4}')

# ??? Cloud / API secret keys ?????????????????????????????????????????
# AWS
_AWS_ACCESS = re.compile(r'\b(A3T[A-Z0-9]|AKIA|AGPA|AIDA|AROA|AIPA|ANPA|ANVA|ASIA)[A-Z0-9]{16}\b')
_AWS_SECRET =
re.compile(r'(?i)(aws[_\-\s]?secret[_\-\s]?access[_\-\s]?key|aws[_\-\s]?secret)[\s:=]+([A-Za-z0-9/+=]{40})',
re.MULTILINE)
_AWS_SESSION = re.compile(r'(?i)(aws[_\-\s]?session[_\-\s]?token)[\s:=]+([A-Za-z0-9/+=]{100,})', re.MULTILINE)

# Generic API / secret key patterns
_API_KEY =
re.compile(r'(?i)(api[_\-\s]?key|secret[_\-\s]?key|private[_\-\s]?key|access[_\-\s]?token|auth[_\-\s]?token|bearer[_\-\s]?t
oken)[\s:=]+([A-Za-z0-9\-\._/+=]{10,})')
```

Multi-PDF RAG Application - Architecture & Codebase Explanation

```
_PASSWORD = re.compile(r'(?i)(password|passwd|pwd)[\s:=]+(\S+)')
_GROQ_KEY = re.compile(r'gsk_[A-Za-z0-9]{48,}')
_OPENAI_KEY = re.compile(r'sk-[A-Za-z0-9]{32,}')
_GITHUB_TOKEN = re.compile(r'gh[pousr]_[A-Za-z0-9]{36,}')

# ??? Personally Identifiable Information (PII) ??????????????????????????????
# SSN (US Social Security Number)
_SSN = re.compile(r'\b\d{3}[-\s]?d{2}[-\s]?d{4}\b')

# Date of Birth ? common formats
_DOB = re.compile(
    r'(?i)(dob|date\s*of\s*birth|birth\s*date)\s*[:=-]?s*'
    r'(\d{1,2}[/-]\d{1,2}[/-]\d{2,4}'
    r'|\d{4}[/-]\d{1,2}[/-]\d{1,2}'
    r'|(?:Jan|Feb|Mar|Apr|May|Jun|Jul|Aug|Sep|Oct|Nov|Dec)[a-z]*\.\?s*\d{1,2},?\?s*\d{4})'
)

# Emergency / personal contact labelled lines
_EMERGENCY_CONTACT = re.compile(
    r'(?i)(emergency\s*contact|next\s*of\s*kin)[\s:=\-]+[^\n]{3,80}'
)

# Insurance / account / record IDs
_INSURANCE_ID =
re.compile(r'(?i)(insurance\s*id|policy\s*no|member\s*id|insured\s*id)\s*[:=-]\s*[A-Za-z0-9\-\-]{5,30}')
_ACCOUNT_NO = re.compile(r'(?i)(account\s*(number|no|#)|acct\s*(no|#)?)[\s:=]+[A-Za-z0-9\-\-]{5,30}')
_IBAN = re.compile(r'\b[A-Z]{2}\d{2}[A-Z0-9]{1,30}\b')
_ROUTING_NO = re.compile(r'(?i)(routing\s*(?:number|no|#))[\s:=]+\d{9}')

# Physical addresses
_ADDRESS = re.compile(
    r'\b\d{1,5}\s+[A-Za-z0-9\.\s]+(?:Street|St|Avenue|Ave|Road|Rd|Boulevard|Blvd|Lane|Ln|Way|Drive|Dr|Court|Ct)\b',
    re.IGNORECASE,
)

# IP Addresses
_IP = re.compile(r'\b(?:\d{1,3}\.){3}\d{1,3}\b')

# Crypto wallet addresses (Bitcoin / Ethereum)
_BTC = re.compile(r'\b[13][a-km-zA-HJ-NP-Z1-9]{25,34}\b')
_ETH = re.compile(r'\b0x[a-fA-F0-9]{40}\b')

def redact_text(text: str) -> str:
    """Redact all detected PII and secret patterns from *text*.
    Returns the sanitised string with placeholders.
    """
    # Cloud / API keys (order matters ? specific first)
    text = _AWS_ACCESS.sub('[REDACTED_AWS_KEY_ID]', text)
```

Multi-PDF RAG Application - Architecture & Codebase Explanation

```
text = _AWS_SECRET.sub(r'\1: [REDACTED_AWS_SECRET]', text)
text = _AWS_SESSION.sub(r'\1: [REDACTED_AWS_SESSION]', text)
text = _GROQ_KEY.sub('[REDACTED_GROQ_KEY]', text)
text = _OPENAI_KEY.sub('[REDACTED_OPENAI_KEY]', text)
text = _GITHUB_TOKEN.sub('[REDACTED_GITHUB_TOKEN]', text)
text = _API_KEY.sub(r'\1: [REDACTED_KEY]', text)
text = _PASSWORD.sub(r'\1: [REDACTED]', text)

# Financial
text = _CARD.sub('[REDACTED_CARD]', text)
text = _CVV.sub(r'\1: [REDACTED_CVV]', text)
text = _IBAN.sub('[REDACTED_IBAN]', text)
text = _ROUTING_NO.sub(r'\1: [REDACTED_ROUTING]', text)
text = _ACCOUNT_NO.sub(r'\1 [REDACTED_ACCOUNT]', text)

# Medical / personal records
text = _SSN.sub('[REDACTED_SSN]', text)
text = _DOB.sub(r'\1: [REDACTED_DOB]', text)
text = _EMERGENCY_CONTACT.sub(r'\1: [REDACTED_CONTACT]', text)
text = _INSURANCE_ID.sub(r'\1: [REDACTED_INSURANCE_ID]', text)

# Contact info
text = _EMAIL.sub('[REDACTED_EMAIL]', text)
text = _PHONE.sub('[REDACTED_PHONE]', text)

# Network / crypto
text = _IP.sub('[REDACTED_IP]', text)
text = _BTC.sub('[REDACTED_BTC_WALLET]', text)
text = _ETH.sub('[REDACTED_ETH_WALLET]', text)

# Physical location
text = _ADDRESS.sub('[REDACTED_ADDRESS]', text)

return text

# Keep backward?compatible alias used by guard_utils
sanitize_text = redact_text
```